Barrutieta Nulutagua Juan Alexadner
Alcala Bejarano Jennifer Paola
Práctica: Lista Circular con record y Java Swing

En esta práctica hicimos un programa en Java que simula la fila de procesos de un planificador por turnos, como el que usa un sistema operativo. Cada proceso tiene un PID, un nombre y una ráfaga en milisegundos. Los procesos se guardan en una lista circular doblemente enlazada que programamos nosotros desde cero, sin usar ArrayList, LinkedList ni arreglos, y el usuario la maneja desde una ventana hecha con el editor visual de NetBeans.

Para separar las responsabilidades dividimos el proyecto en dos paquetes. En BackEnd quedaron el record Proceso, la clase Nodo y la clase ListaCircular. El record solo se encarga de guardar los datos de un proceso y de revisar que no sean inválidos; como es inmutable, una vez creado ya no se puede cambiar. El Nodo solo guarda un Proceso y sus dos enlaces, siguiente y anterior. La ListaCircular es la única que sabe cómo se conectan los nodos y ofrece métodos para insertar, remover, mostrar, buscar y navegar. En FrontEnd está la ventana, que se armó arrastrando y soltando componentes en el GUI Builder. La ventana nunca toca un nodo: solo lee los campos de texto, valida lo que escribió el usuario, crea el record y llama a los métodos de la lista. Si la lista detecta un problema, como que esté vacía o que la posición no exista, lanza una excepción con un mensaje y la ventana lo muestra en un JOptionPane. Así, si mañana cambiamos la ventana por otra, la lista sigue funcionando igual.

La navegación circular salió casi sola gracias a cómo armamos el anillo. Cada vez que insertamos o removemos, nos aseguramos de que el siguiente del último sea el primero y de que el anterior del primero sea el último. La lista guarda además un nodo actual, que es el que se ve en pantalla. Cuando el usuario presiona Siguiente, el actual simplemente pasa a su enlace siguiente, y cuando presiona Anterior pasa a su enlace anterior. Como el último está conectado con el primero, al avanzar desde el último aparece el primero sin tener que escribir un caso especial, y al retroceder desde el primero aparece el último. La posición que se muestra en pantalla se calcula con el residuo de dividir entre el tamaño, para que también dé la vuelta de la última posición a la cero y al revés.

En una lista circular ningún enlace es nulo, así que no se puede recorrer esperando encontrar un null como en una lista normal, porque el ciclo nunca terminaría. Para evitarlo, la búsqueda por valor empieza en el primero, revisa el PID de cada nodo y avanza, y se detiene en cuanto vuelve a llegar al primero. Eso significa que da exactamente una vuelta. Si en esa vuelta no

encontró el PID, regresa que no existe y la ventana avisa que se recorrió todo el anillo sin encontrarlo. Para llegar a una posición usamos un contador que va de cero hasta el número pedido, y además elegimos si conviene caminar desde el primero hacia adelante o desde el último hacia atrás, según qué extremo esté más cerca.

Cuando se elimina el primero en una lista con varios elementos, primero se toma su vecino de atrás, que es el último, y su vecino de adelante, que será el nuevo primero. Luego el último apunta con su siguiente al nuevo primero, y el nuevo primero apunta con su anterior al último. Al final la referencia primero se mueve a ese nuevo nodo. Para eliminar el último se hace lo mismo pero al revés: su vecino de atrás pasa a ser el nuevo último, ese nuevo último apunta con su siguiente al primero, el primero apunta con su anterior al nuevo último y la referencia ultimo se mueve. En los dos casos el nodo que sale queda apuntándose a sí mismo para que ya no tenga relación con el anillo. Si la lista solo tenía un nodo, simplemente se ponen primero y último en nulo y la lista queda vacía.

La ventaja principal de la lista circular frente a una lista lineal es que no tiene un final donde haya que detenerse o regresar al inicio a mano. En un carrusel de opciones o en un planificador por turnos siempre se quiere pasar al siguiente, y cuando se acaba la fila se vuelve a empezar; con la lista circular eso ocurre con solo seguir el enlace, sin preguntar si estamos en el último ni volver a buscar el primero. También se puede llegar al primero y al último de inmediato desde cualquiera de los dos extremos, y al tener enlaces dobles se puede ir hacia atrás con la misma facilidad. Además no se desperdicia memoria en referencias nulas y el código de la navegación queda más corto y con menos casos especiales.

Para comprobar que todo funcionaba hicimos pruebas con un solo nodo, donde vimos que el siguiente y el anterior apuntan al mismo nodo y que primero y último son el mismo. También comprobamos que avanzar desde el último muestra el primero, que retroceder desde el primero muestra el último, que después de cada inserción y remoción el anillo sigue cerrado, y que intentar operar con la lista vacía, con posiciones fuera de rango, con un PID que no existe o con letras en lugar de números muestra un aviso en lugar de que el programa truene. En la parte de abajo de la ventana dejamos una etiqueta que muestra a qué PID apunta el siguiente del último y el anterior del primero, para poder ver en todo momento que el anillo está bien conectado.